

Project Design Phase-II

Technology Stack (Architecture & Stack)

Date	30 September 2026
Team ID	SWTID-2026-1429
Project Name	ComicCraft - AI Comic Story Creator using Gemini Models
Maximum Marks	4 Marks

Technical Architecture:

Architecture of ComicCraft: the user submits story details in the browser, the FastAPI backend calls Gemini Flash and Gemini Pro for the story and Stable Diffusion for the images, then builds the layout and exports the PDF.

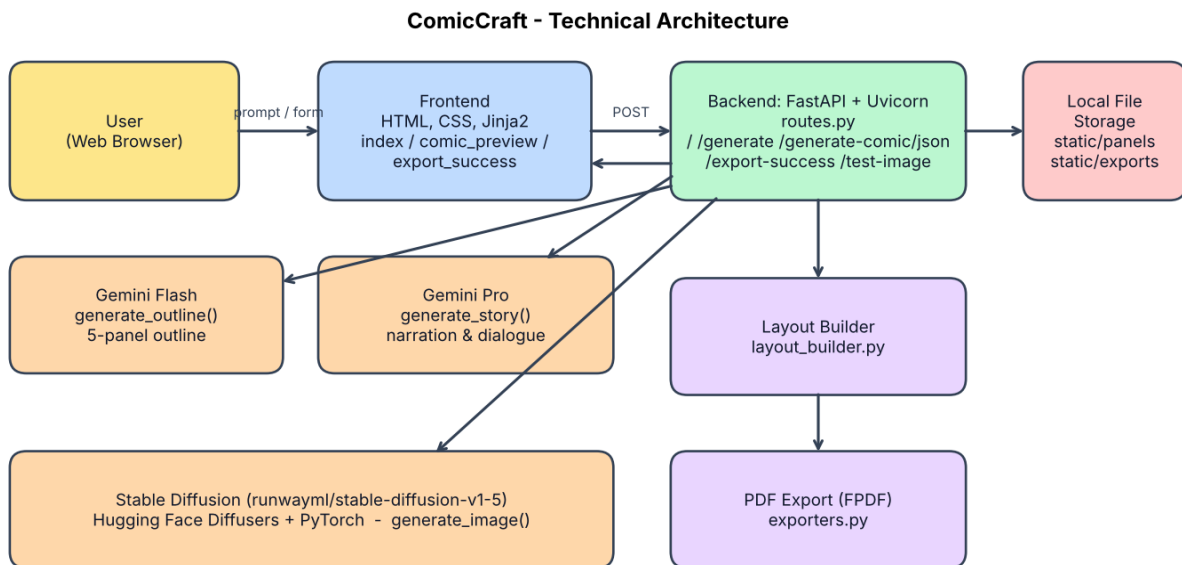


Table-1 : Components & Technologies:

S.No	Component	Description	Technology
1	User Interface	Web UI where users enter the story prompt, main character name, setting, tone and art style, then preview the generated comic panel by panel and download it as a PDF.	HTML, CSS, Jinja2 templates (index.html, comic_preview.html, export_success.html)
2	Application Logic-1	FastAPI backend that handles routing and user input: form data via Form(...) and JSON via a Pydantic PromptRequest model, error handling with HTTPException.	Python, FastAPI, Uvicorn (routes.py)

S.No	Component	Description	Technology
3	Application Logic-2	Story generation: generate_outline() creates a structured 5-panel outline; generate_story() writes narration and character dialogue for each panel.	Google Gemini API: Gemini Flash (gemini-1.5-flash), Gemini Pro (gemini-1.5-pro)
4	Application Logic-3	Comic assembly and export: build_comic_layout() matches each image with its panel text; save_pdf() compiles the multi-page PDF with a timestamped filename.	Python, FPDF, Pillow (layout_builder.py, exporters.py)
5	Database	No persistent database in the current version; comic data is held per request as Python lists of dictionaries (panel number, image path, text).	In-memory Python data structures
6	Cloud Database	Not used in the current version. A cloud database is a possible future addition for user profiles and comic libraries.	Not applicable (future scope)
7	File Storage	Generated panel images are saved to static/panels and exported PDFs to static/exports on the server.	Local filesystem
8	External API-1	Gemini API for outline, narration and dialogue generation. The API key is read from the GEMINI_API_KEY environment variable.	Google Generative AI API (google-generativeai)
9	External API-2	Hugging Face access for downloading the Stable Diffusion model weights, using the HF_API_KEY token.	Hugging Face Hub / Diffusers
10	Machine Learning Model	Stable Diffusion generates a comic-style illustration for each panel from the AI-written image prompt.	runwayml/stable-diffusion-v1-5 via Hugging Face Diffusers, PyTorch, Transformers, Accelerate
11	Infrastructure (Server / Cloud)	Application deployed on a local system in a Python virtual environment and served by Uvicorn at http://127.0.0.1:8000 (API docs at /docs). Local Server Configuration: Python, pip, venv, requirements.txt, .env file for API keys. Cloud Server Configuration: not used yet; cloud deployment is planned as future work.	Local (Uvicorn ASGI server)

Table-2: Application Characteristics:

S.No	Characteristics	Description	Technology
1	Open-Source Frameworks	Open-source frameworks and libraries used across the stack.	FastAPI, Uvicorn, Jinja2, Hugging Face Diffusers, Transformers, PyTorch, FPDF, Pillow, python-dotenv
2	Security Implementations	API keys kept out of the code in environment variables or a .env file; input validated with Pydantic and FastAPI Form parameters; prompts sanitized for safe filenames; try-except blocks and HTTPException for controlled error handling.	Environment variables, python-dotenv, Pydantic
3	Scalable Architecture	Modular three-part design (frontend, FastAPI backend, AI integration) with each function in its own module, so components can be extended or moved to separate services later.	FastAPI, Python modules, ASGI
4	Availability	Current version runs as a single local instance. The backend keeps no user sessions, so it can later be run as multiple instances behind a load balancer on a cloud platform.	Uvicorn (cloud deployment planned)
5	Performance	Gemini Flash is used for fast structured outlines and Gemini Pro only for detailed narration; the ASGI server handles requests efficiently; Accelerate and PyTorch support faster image generation; images are saved to disk for reuse in the preview and PDF. No cache or CDN in the current version.	Uvicorn, Gemini Flash, PyTorch, Accelerate

References:

<https://c4model.com/>

<https://www.ibm.com/cloud/architecture>

<https://aws.amazon.com/architecture>

<https://fastapi.tiangolo.com/>

<https://ai.google.dev/>

<https://huggingface.co/docs/diffusers/en/index>